Rapport de conception

Plateforme de visionnage vidéo synchronisé · Projet IL 2026-2027, semestre 1

Prestataire	Groupe A : Daniel Court, Emilien Jouberjean, Youness Nizari, Hatim Chmali
Client	Responsable de l'UE Projet IL, Université d'Avignon (L3 Informatique)
Version	1.0, novembre 2026 · fait suite aux spécifications d'analyse v1.0
Contenu	Architecture, modules et répartition, choix techniques, synchronisation et prototype, gestion de projet, Gantt, liens

1. Objet

Ce rapport explique **comment** le produit décrit dans l'analyse sera construit : architecture, découpage en modules, choix techniques justifiés et limites, conception de la synchronisation, tests prévus. Il contient aussi tout ce qui concerne la gestion de projet : répartition du travail, diagramme de Gantt, méthode de suivi et outils, avec les liens correspondants (section 7).

2. Architecture

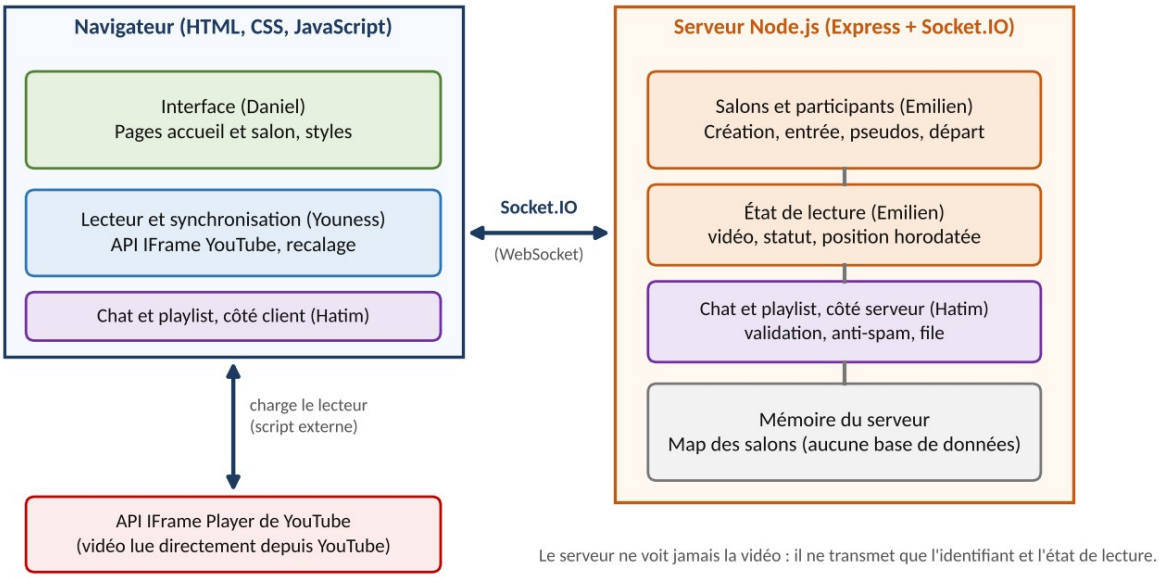


Figure 1 – Architecture client-serveur. Une couleur par responsable.

L'application est un **site web servi par un serveur Node.js**. Le navigateur affiche l'interface et le lecteur YouTube ; le serveur ne lit jamais la vidéo, il garde seulement l'état de chaque salon et le diffuse. Toute la communication passe par une connexion Socket.IO ouverte par chaque navigateur, qui est regroupée en « room » par salon : un message émis pour un salon n'atteint que ses participants (VID-04). Le serveur est la **source de vérité** : en cas de doute, c'est son état qui l'emporte.

3. Modules et interfaces

Le découpage suit les rôles du cahier des charges. Chaque module ne dépend des autres que par les événements de l'analyse (section 5), regroupés dans un seul fichier de constantes `shared/events.js` ; c'est ce qui permet de développer en parallèle puis d'intégrer en deux semaines.

Module	Responsable	Rôle	Fichiers principaux
Pages et interface	Daniel	Page d'accueil (pseudo, créer, rejoindre), page du salon (lecteur, chat, playlist, participants), styles, affichage des erreurs.	`client/index.html`, `client/salon.html`, `client/css/`, `client/js/ui.js`
Lecteur et synchronisation	Youness	Intégration de l'API YouTube, chargement, envoi des actions, recalage, bouton de lecture si blocage.	`client/js/player.js`, `client/js/sync.js`, `shared/youtube.js`
Chat et playlist (client)	Hatim	Zone de saisie et d'affichage des messages, liste de la playlist, boutons ajouter et suivant.	`client/js/chat.js`, `client/js/playlist.js`
Salons et serveur	Emilien	Serveur Express et Socket.IO, création, entrée et départ, hôte, reconnexion, isolation par salon.	`server/index.js`, `server/rooms.js`
État de lecture	Emilien	Vidéo, statut, position horodatée, calcul de la position courante (RG6).	`server/playback.js`
Chat, pseudonymes et playlist (serveur)	Hatim	Validation des pseudonymes et des messages, limitation de cadence, file de playlist.	`server/validators.js`, `server/chat.js`, `server/playlist.js`
Événements communs	Emilien (tenue), tous (relecture)	Noms et formats des événements, constantes (limites, codes d'erreur).	`shared/events.js`, `shared/limits.js`

Relations entre modules. La page du salon (Daniel) offre trois zones identifiées (`#lecteur`, `#chat`, `#playlist`) dans lesquelles les modules de Youness et de Hatim s'affichent. `rooms.js` reçoit les événements et les confie à `playback.js`, `chat.js` ou `playlist.js` ; ceux-ci ne connaissent pas Socket.IO : ils reçoivent des données et renvoient un résultat, ce qui les rend testables sans réseau.

4. Choix techniques

Élément	Choix	Justification et limites
Langage	JavaScript partout	Un seul langage pour toute l'équipe : chacun peut relire et remplacer un autre membre.
Interface	HTML, CSS et JavaScript, sans framework	Seulement deux pages ; un framework alourdirait la prise en main. L'interface reste en JavaScript « module » découpé par fichier.
Serveur	Node.js et Express	Même langage que le navigateur ; Express sert les pages et héberge Socket.IO.
Temps réel	Socket.IO	Fournit les rooms, la reconnexion automatique et un repli si WebSocket est bloqué, ce qui évite de les programmer à la main.
Lecteur	API YouTube IFrame Player	Voie officielle pour lire et piloter une vidéo. **Limite** : elle ne signale pas directement un déplacement dans la vidéo (voir section 5).
Stockage	Mémoire du serveur (une `Map` de salons)	Un salon est éphémère, aucune base à gérer. **Limite connue** : un redémarrage du serveur supprime tous les salons ; les clients affichent alors « salon introuvable » et peuvent en recréer un. Si cette limite devient gênante (comptes, historique au semestre 2), on utilisera la base de données de l'université.

Élément	Choix	Justification et limites
Tests	`node:test` (inclus dans Node.js) et `socket.io-client`	Aucune dépendance en plus ; le second permet de simuler plusieurs navigateurs dans un test d'intégration.
Qualité du code	ESLint, Prettier, README	Style commun pour faciliter la relecture croisée.

5. Conception de la synchronisation

C'est le risque technique principal ; on le traite avant de coder le reste.

- **État de référence.** Le serveur garde `videoId`, `statut`, `position` et `majLe` (heure du serveur au dernier changement). Position courante = `position` + (maintenant − `majLe`) si lecture (RG6).
- **Horloges.** Les horloges des navigateurs diffèrent. À l'arrivée, le client mesure son décalage avec le serveur en échangeant quelques messages (aller-retour divisé par deux, moyenne de 5 mesures) et corrige `heureServeur` en conséquence.
- **Recalage.** Toutes les 5 s, le client compare la position de son lecteur à la position attendue ; si l'écart dépasse 2 s, il se replace (`seekTo`).
- **Pas d'écho.** Un changement provoqué par le programme (recalage, arrivée d'un `video:etat`) lève un drapeau qui empêche d'émettre un `video:action` ; sinon deux navigateurs se renverraient la même action sans fin.
- **Déplacement dans la vidéo.** L'API n'émet pas d'événement quand l'utilisateur déplace la barre de progression. Le client relève la position toutes les 500 ms ; un saut de plus de 1,5 s qui n'est pas dû à la lecture normale est traité comme un `deplacer`.
- **Lecture bloquée.** Si le navigateur refuse la lecture automatique (état du lecteur qui ne passe pas en lecture après 1,5 s), un bouton « Lancer la lecture » s'affiche.
- **Fin de vidéo.** Chaque client signale `video:fin` ; le serveur ne tient compte que du premier signalement pour une vidéo donnée, sinon la playlist avancerait autant de fois qu'il y a de participants.

Prototype jetable. Pendant la conception, Youness réalise une page unique et un serveur d'une soixantaine de lignes (un salon, un lecteur, boutons lecture, pause, déplacement) pour vérifier ces mécanismes. Critère de réussite : sur deux navigateurs (Chrome et Firefox), l'écart reste inférieur à 2 s sur 10 actions successives. Le code du prototype n'est pas réutilisé tel quel. Résultat : [à compléter par Youness après l'essai : écarts relevés, problèmes rencontrés, ajustements de l'algorithme].

6. Tests prévus

Niveau	Contenu	Responsable et outil
Unitaire	Validation des pseudonymes, messages et URL ; calcul de position (RG6) ; file de playlist ; règles RG1 à RG5.	Chaque auteur pour son module, avec `node:test`.
Intégration	Plusieurs clients simulés sur un vrai serveur : création, arrivée en cours de lecture, diffusion, isolation de deux salons, salon vide fermé.	Emilien (serveur), Youness (scénarios de synchronisation).
Recette manuelle	Les cinq scénarios 4.1 à 4.5 du cahier des charges, sur Chrome et Firefox avec au moins trois participants dans deux salons ; utilisabilité jouée par un membre qui n'a pas fait l'interface.	Toute l'équipe ; résultats dans le tableau de tests (date, cas, navigateur et version, résultat, anomalie).

7. Gestion de projet

7.1 Méthode

Le projet suit le **modèle en cascade** : chaque phase (besoins, analyse, conception, réalisation, tests, bilan) est validée avant la suivante, et le développement ne commence qu'après validation de la conception. Ce choix convient à un périmètre court et connu à l'avance, avec un client unique et des livrables datés. Son risque, découvrir tard un problème technique, est réduit par le prototype de synchronisation fait pendant la conception, et par deux semaines d'intégration (16 au 29 novembre).

7.2 Répartition du travail

Membre	Responsabilité	Exigences portées	Relecteur	Heures prévues
Daniel Court	Pages, interface, styles	SAL-03 et CHAT-01 (affichage), utilisabilité	Hatim Chmali	64 h
Emilien Jouberjean	Serveur, salons, état partagé	SAL-01 à SAL-05, VID-04, état de VID-01 à VID-03	Youness Nizari	64 h
Youness Nizari	Lecteur, synchronisation client	VID-01 à VID-03, VID-05, prototype	Emilien Jouberjean	64 h
Hatim Chmali	Chat, pseudonymes, playlist	CHAT-01, CHAT-02, PLAY-01 à PLAY-03	Daniel Court	64 h

Chaque fonctionnalité a donc un développeur et un relecteur : personne n'est seul sur une partie, et chacun peut expliquer le travail d'un autre à la soutenance. Les 64 h par personne (256 h pour le groupe) se répartissent ainsi :

Membre	Analyse et conception	Réalisation et intégration	Tests et corrections	Documentation et soutenance
Daniel	14 h	39 h	7 h	4 h
Emilien	14 h	41 h	5 h	4 h
Youness	14 h	39 h	7 h	4 h
Hatim	14 h	39 h	7 h	4 h
Groupe	**56 h**	**158 h**	**26 h**	**16 h**

Les 14 h d'analyse et conception de chacun se répartissent entre les trois rendus et le prototype : Daniel, maquettes des écrans, cas d'utilisation et conception des pages ; Emilien, modèle de données, règles de gestion et architecture du serveur ; Youness, séquence de synchronisation, prototype et conception du lecteur ; Hatim, séquences du chat et de la playlist, validations et conception de ses modules. Tous relisent le cahier des charges, vérifient la cohérence des trois documents et préparent le rendu (environ 2 h chacun).

7.3 Diagramme de Gantt

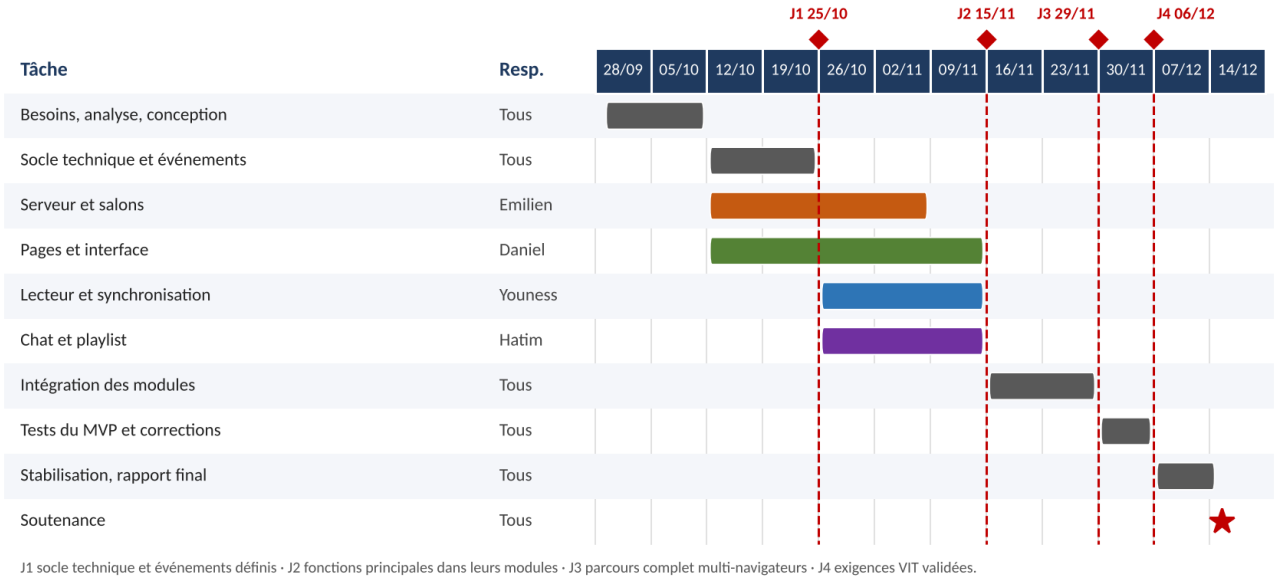


Figure 2 – Planning du projet. Une couleur par membre pour ses tâches ; gris pour les tâches communes ; losanges rouges : jalons.

Jalons : 25 octobre, socle technique et événements définis · 15 novembre, fonctions principales réalisées dans leurs modules · 29 novembre, parcours complet utilisable sur plusieurs navigateurs · 6 décembre, exigences VIT validées ou écarts documentés. Les fonctionnalités IMP et MIN ne sont engagées qu'après validation du MVP ; la semaine du 7 décembre sert de marge.

7.4 Suivi, outils et règles de travail

- **Réunions** : point de 20 minutes avec l'enseignant à chaque séance ; réunion d'équipe chaque semaine, avec un compte rendu daté des décisions et des actions (où l'on décide aussi toute modification d'un événement du contrat).
- **Tableau Trello** : une carte par tâche, rattachée à une exigence (SAL-01…) et à un membre, qui avance par les colonnes À faire, En cours, En relecture, Terminé.
- **GitHub** : branche principale protégée ; une branche par fonctionnalité ; fusion seulement après relecture par le relecteur désigné ; un commit correspond à une tâche, avec un message qui reprend l'identifiant de l'exigence (par exemple « CHAT-01 : affichage du pseudonyme »).
- **Feuille d'heures partagée** : chacun note en temps réel la date, la tâche, le travail effectué, le temps passé et le lien vers le commit. Les heures réelles sont comparées aux estimations.
- **Écarts** : tout écart au cahier des charges, aux estimations ou au Gantt est conservé avec sa cause et nourrit le bilan critique du rapport final.

7.5 Risques

Risque	Réponse prévue
Synchronisation plus difficile que prévu.	Prototype pendant la conception, tolérance de 2 s, recalage automatique ; position de repli : synchroniser seulement lecture et pause.
Lecture automatique bloquée ou vidéo non intégrable.	Bouton pour lancer la lecture, message d'erreur clair, page non bloquée.
Retard sur le planning.	Les fonctionnalités MIN, puis IMP, sont abandonnées en premier, jamais les VIT ; semaine tampon du 7 décembre.
Membre moins disponible.	Relecture croisée, réaffectation décidée en réunion et documentée.
Modules incompatibles à l'intégration.	Contrat d'événements figé avant la réalisation, tests d'intégration, deux semaines d'intégration.

Risque	Réponse prévue
Perte des salons au redémarrage du serveur.	Limite acceptée pour le MVP et expliquée à l'utilisateur ; base de données de l'université si besoin au semestre 2.

7.6 Liens

Outil	Lien
Dépôt GitHub	[lien du dépôt GitHub à ajouter]
Suivi des heures de travail	[lien de la feuille d'heures partagée à ajouter]
Tableau Trello	[lien du tableau Trello à ajouter]
Comptes rendus de réunion	[lien du dossier des comptes rendus à ajouter]